

compile_pg

NAME

compile_pg
Creates a power and ground network including straps, rings, macro connections, standard cell connections and via connections based on the specified power ground strategies.

SYNTAX

```
status compile_pg
  [-strategies strategy_list]
  [-via_rule rule_list]
  [-ignore_drc]
  [-ignore_via_drc]
  [-show_phantom]
  [-undo]
  [-tag tag_name]
  [-create_ml_data]
  [-use_ml_model]
```

Data Types

```
strategy_list  list
rule_list      list
tag_name       string
```

ARGUMENTS

- strategies strategy_list**
Creates a power network based only on the specified list of strategies. By default, the tool creates a power ground network using all strategies which have been defined.
- via_rule rule_list**
Specifies the via rule list between strategies and existing shapes. The via rule list rule_list contains strategy-level via rules defined by the set_pg_strategy_via_rule command. If no strategy via rule is specified, the default via defined in the technology file is created at all intersections between any two orthogonal shapes on different layers.
- ignore_drc**
Creates the power network while ignoring DRCs for both wires and vias. By default, all DRCs are considered when creating the power network. This option is mutually exclusive with the -ignore_via_drc option.
- ignore_via_drc**
Creates the power network while ignoring via DRCs. By default, all DRCs are considered when creating the power network. When this option is set, DRCs for wires are checked and fixed, DRCs within each stacked via are checked and fixed. Only DRCs between stacked vias and neighboring objects are ignored. This option is mutually exclusive with -ignore_drc option.
- show_phantom**
Generates report for wires/vias that should be created but failed due to fixing DRC violation. Detailed information will be displayed in error browser. The report includes two parts, phantom wire and phantom via. Wire information and DRC violation are included in one phantom wire. Only DRC violations that cut part of wire shape while fixing will be included in phantom wire. Abutted phantom wires will be merged and reported together. Via information and DRC violation are included in one phantom via. Via information includes top/bottom wire information, via def, net name. DRC violation shows initial violation before fixing. If -ignore_drc or -ignore_via_drc is specified, phantom via will not be generated.
- undo** Removes the power and ground network created by the most recent compile_pg command.
- tag tag_name**
Specifies a tag name to assign to all new vias and shapes created by this command. This tag name is used as contents of an user attribute tag for filtering collections at later stages (see the example in the EXAMPLES section). By default, no tags are assigned.

-create_ml_data
This option enables this command to create training data for training ML model for fixability prediction. When this option is set, the command only creates ML training data, not creating PG wires/vias.

-use_ml_model
This option enables this command to load pre-trained ML model for fixability prediction.

DESCRIPTION

This command creates the power network based on the specified power ground strategies.

Signal routes by default are ignored for DRC checking and correction during PG creation. Use the following app option to honor signal route DRC during PG creation.

```
prompt> set_app_options -list {plan.pgroute.honor_signal_route_drc true}
```

EXAMPLES

The following example creates a power ground network using only the example1 and example2 strategies. The rule1 via rule is defined by the set_pg_strategy_via_rule command.

```
prompt> compile_pg -strategies {example1 example2}\  
-via_rule rule1
```

The following example compiles strategy example1 using 4 threads defined in the global host_options mt for via creation.

```
prompt> set_host_options -max_cores 4  
prompt> compile_pg -strategies {example1}
```

The following example creates collections of vias and wires that are tagged with the name pg1.

```
prompt> compile_pg -tag pg1  
prompt> set vias [get_vias -filter tag==pg1]  
prompt> set wires [get_shapes -filter tag==pg1]
```

The following example creates ML training data in memory, output training data to files in the directory ./pg_model, and train ML model based on the training data. The trained ML model is stored in the directory ./pg_model.

```
prompt> compile_pg -create_ml_data  
prompt> train_pg_ml_model
```

The following example creates ML training data in memory, output training data to files in directory pg_data_1, and train ML model based on the training data. The trained ML model is stored in the directory ./pg_model.

```
prompt> compile_pg -create_ml_data  
prompt> create_pg_ml_data -output_directory pg_data_1  
prompt> train_pg_ml_model -input_directory {pg_data_1}
```

The following example loads pre-trained ML model in the directory ./pg_model for fixability prediction to reduce runtime of PG via creation.

```
prompt> compile_pg -use_ml_model
```

In addition to the above examples, see the Examples page in the PG Planning section of the Task Assistant for more information. To view the Examples page, start the GUI and invoke the following command: gui_show_task_assistant -task "Design Planning:PG Planning->Examples->Overview".

SEE ALSO

[remove_pg_strategies\(2\)](#)
[report_pg_strategies\(2\)](#)
[report_pg_strategy_via_rules\(2\)](#)
[set_host_options\(2\)](#)
[set_pg_strategy\(2\)](#)
[set_pg_strategy_via_rule\(2\)](#)
[get_vias\(2\)](#)
[get_shapes\(2\)](#)
[create_pg_ml_data\(2\)](#)
[train_pg_ml_model\(2\)](#)

